Part 2

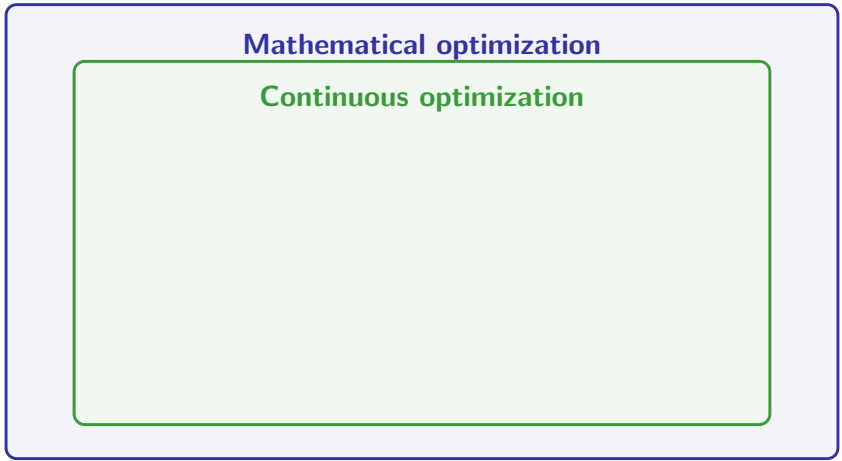
(40 min)

A brief introduction to the SDP formulation of the Performance Estimation problem

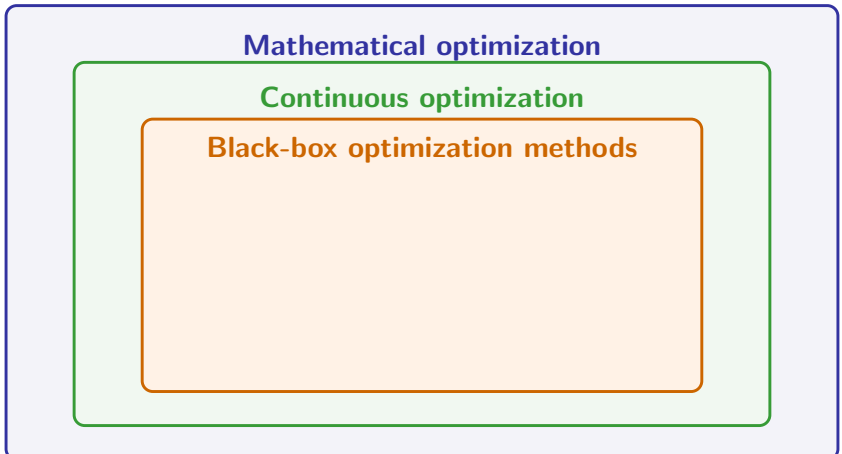
Context

Mathematical optimization

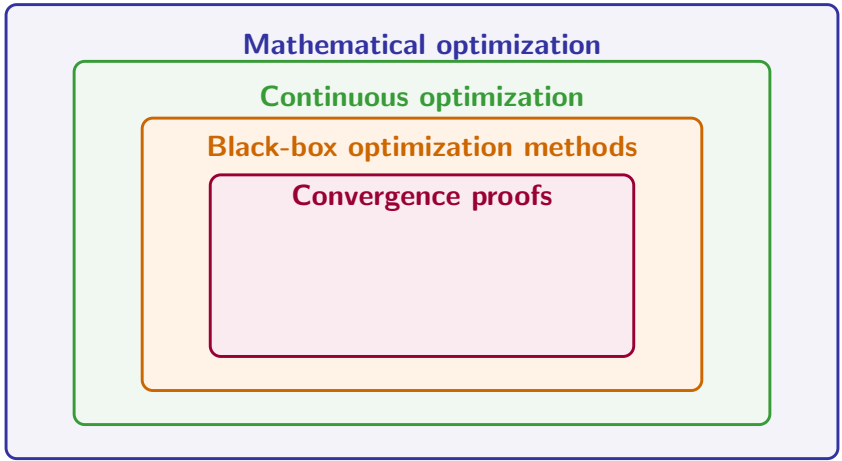
Context



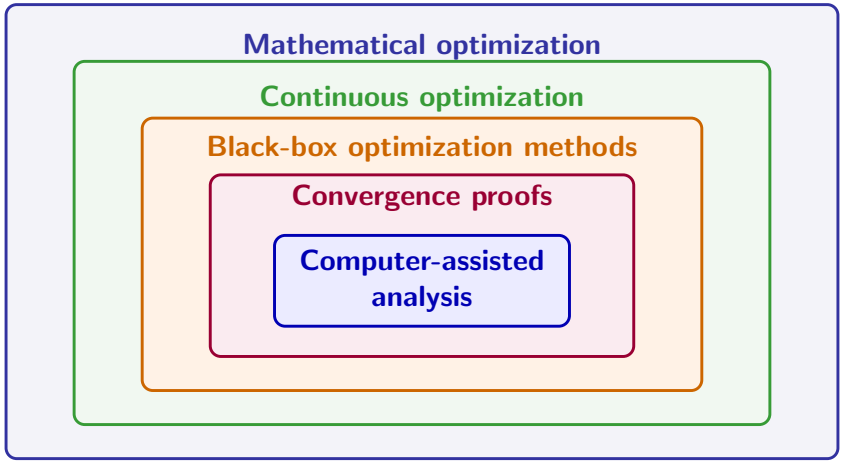
Context


Mathematical optimization
Continuous optimization
Black-box optimization methods

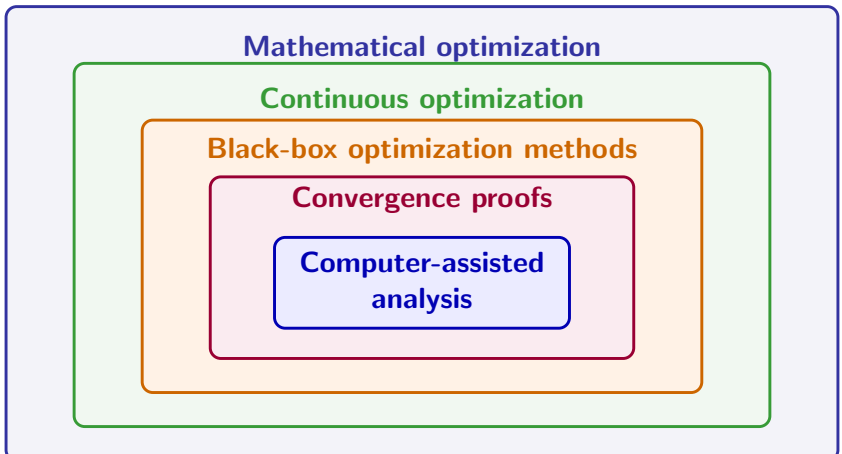
Context


Mathematical optimization
Continuous optimization
Black-box optimization methods
Convergence proofs

Context


Mathematical optimization
Continuous optimization
Black-box optimization methods
Convergence proofs
Computer-assisted analysis

Context



Our topic today: **Performance Estimation Problem = PEP**, a methodology for computer-assisted convergence analysis of black-box optimization methods

Outline

Background: black-box first-order optimization methods

Optimization problem

$$\min_{x \in \mathbb{R}^d} f(x)$$

- f belongs to some function class
- dimension d may be very large

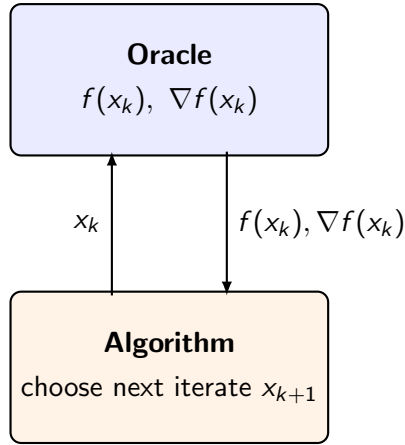
Black-box oracle model

Algorithm does not know f explicitly
At iteration k query a point x_k and receive $f_k = f(x_k)$ and $g_k = \nabla f(x_k)$

General first-order method

$$x_{k+1} = \mathcal{M}_k(x_0, g_0, g_1, \ldots, g_k)$$

using only previously observed info



Goal

Prove convergence rates such as

$$f(x_N) - f_* \leq \frac{LR^2}{2N}$$

Background: examples of function classes

- L-**smooth nonconvex functions** = gradient Lipschitz $\quad \to \quad \mathcal{F}_{-L,L}$

$$\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\| \quad \forall x, y$$

- L-**smooth convex functions** $\quad \to \quad \mathcal{F}_{0,L}$

$$f(y) \ge f(x) + \langle \nabla f(x), y - x\rangle, \quad \|\nabla f(x) - \nabla f(y)\| \le L\|x - y\| \quad \forall x, y$$

- μ-**strongly convex functions** $\quad \to \quad \mathcal{F}_{\mu,+\infty}$

$$f(y) \ge f(x) + \langle \nabla f(x), y - x\rangle + \frac{\mu}{2}\|y - x\|^2$$

or equivalently

$$x \mapsto f(x) - \frac{\mu}{2}\|x\|^2 \quad \text{is convex.}$$

- ρ-**weakly convex functions** $\quad \to \quad \mathcal{F}_{-\rho,+\infty}$

$$x \mapsto f(x) + \frac{\rho}{2}\|x\|^2 \quad \text{is convex.}$$

Background: examples of function classes

- All of the above can be seen as special cases of the general class of L-**smooth** μ-**strongly convex functions** $\mathcal{F}_{\mu,L}$ where $-\infty \leq \mu \leq L \leq +\infty$

- When f is $\mathcal{C}^2$: equivalent to bounds on Hessian spectrum

$$\mu \quad \leq \quad \lambda_k(\nabla^2 f(x)) \quad \leq L \qquad \forall k, x$$

- **Convex functions with bounded subgradients**

$$f(y) \geq f(x) + \langle g_x, y - x \rangle \quad \forall x, y, \quad \forall g_x \in \partial f(x),$$

together with

$$\|g_x\| \leq B \quad \forall x, \quad \forall g_x \in \partial f(x).$$

which is equivalent to f being B-Lipschitz

$$|f(x) - f(y)| \leq B\|x - y\| \quad \forall x, y.$$

Background: classical examples of convergence results

(1) Smooth convex minimization

Assume f is convex with L-Lipschitz gradient.
Gradient method with stepsize $1/L$:

$$x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k).$$

Then

$$f(x_N) - f_* \leq \frac{L\|x_0 - x_*\|^2}{2N}.$$

$$\boxed{f(x_N) - f_* = O(1/N)}$$

Background: classical examples of convergence results

(2) Smooth nonconvex minimization

Assume f has L-Lipschitz gradient.
Gradient method with stepsize $1/L$:

$$x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k).$$

Then

$$\min_{0\le k\le N} \|\nabla f(x_k)\|^2 \le \frac{2L(f(x_0) - f_*)}{N+1}.$$

$$\boxed{\min_{0\le k\le N} \|\nabla f(x_k)\|^2 = O(1/N)}$$

Background: classical examples of convergence results

(3) Accelerated gradient method

Assume f is convex with L-Lipschitz gradient.
Nesterov's accelerated fast gradient method

$$\begin{cases} y_k &= x_k + \frac{k-1}{k+2}(x_k - x_{k-1}) \\ x_{k+1} &= y_k - \frac{1}{L}\nabla f(y_k) \end{cases} \qquad \text{(with } x_{-1} = x_0\text{)}$$

achieves

$$f(x_N) - f_* \leq \frac{2L\|x_0 - x_*\|^2}{(N+1)^2}.$$

$$\boxed{f(x_N) - f_* = O(1/N^2)}$$

$O(1/N^2)$ is **optimal** for black-box first-order methods on smooth convex functions,

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

NAGD converges rate $\mathcal{O}\left(\frac{1}{k^2}\right)$ proof 1/6 **Stage 1: make use of convexity & smoothness**

- f cvx: $(\forall \boldsymbol{x} \forall \boldsymbol{y})\Big\{f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \langle \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x}\rangle\Big\}$ gives

$$-f(\boldsymbol{y}) \leq -f(\boldsymbol{x}) + \langle \nabla f(\boldsymbol{x}), \boldsymbol{x} - \boldsymbol{y}\rangle \tag{5}$$

- f L-smooth $(\forall \boldsymbol{a} \forall \boldsymbol{b})\Big\{f(\boldsymbol{a}) - f(\boldsymbol{b}) \leq \langle \nabla f(\boldsymbol{b}), \boldsymbol{a} - \boldsymbol{b}\rangle + \frac{L}{2}\|\boldsymbol{a} - \boldsymbol{b}\|_2^2\Big\}$, with $\boldsymbol{a} = \boldsymbol{x} - \frac{1}{L}\nabla f(\boldsymbol{x}), \boldsymbol{b} = \boldsymbol{x}$,

$$f\Big(\boldsymbol{x} - \frac{1}{L}\nabla f(\boldsymbol{x})\Big) - f(\boldsymbol{x}) \;\leq\; -\frac{1}{L}\|\nabla f(\boldsymbol{x})\|_2^2 + \frac{1}{2L}\|\nabla f(\boldsymbol{x})\|_2^2 = \frac{-1}{2L}\|\nabla f(\boldsymbol{x})\|_2^2. \tag{6}$$

- (5) + (6) will cancel $-f(\boldsymbol{x})$ and give

$$f\Big(\boldsymbol{x} - \frac{1}{L}\nabla f(\boldsymbol{x})\Big) - f(\boldsymbol{y}) \;\leq\; \frac{-1}{2L}\|\nabla f(\boldsymbol{x})\|_2^2 + \langle \nabla f(\boldsymbol{x}), \boldsymbol{x} - \boldsymbol{y}\rangle. \tag{7}$$

- Put $\boldsymbol{x} = \boldsymbol{x}_k$, $\boldsymbol{y} = \boldsymbol{x}^*$ in (7)

$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f^* \;\leq\; \frac{-1}{2L}\|\nabla f(\boldsymbol{x}_k)\|_2^2 + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x}_k - \boldsymbol{x}^*\rangle. \tag{8}$$

- put $\boldsymbol{x} = \boldsymbol{x}_k$, $\boldsymbol{y} = \boldsymbol{y}_k$ in (7)

$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f(\boldsymbol{y}_k) \;\leq\; \frac{-1}{2L}\|\nabla f(\boldsymbol{x}_k)\|_2^2 + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x}_k - \boldsymbol{y}_k\rangle. \tag{9}$$

- Proof overview: (8), (9) link $f(\boldsymbol{y}_{k+1}), f(\boldsymbol{y}_k)$ and f^*. We see $\nabla f(\boldsymbol{x}_k)$ appear in (8), (9) but not in the convergence result, so we eliminate $\nabla f(\boldsymbol{x}_k)$ in (8), (9).

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

Proof 2/6 **Stage 2: eliminate gradient**

$$\boldsymbol{y}_{k+1} = \boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k) \tag{1}$$
$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f^* \le \frac{-1}{2L}\|\nabla f(\boldsymbol{x}_k)\|_2^2 + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x}_k - \boldsymbol{x}^*\rangle. \tag{8}$$
$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f(\boldsymbol{y}_k) \le \frac{-1}{2L}\|\nabla f(\boldsymbol{x}_k)\|_2^2 + \langle \nabla f(\boldsymbol{x}_k), \boldsymbol{x}_k - \boldsymbol{y}_k\rangle. \tag{9}$$

- Simplify notation, let $\delta_k := f(\boldsymbol{y}_k) - f^*$, then

$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) \overset{(1)}{=} f(\boldsymbol{y}_{k+1}) \tag{10}$$
$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f^* \overset{(10),\delta_k}{=} \delta_{k+1} \tag{11}$$
$$f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f(\boldsymbol{y}_k) = f\Big(\boldsymbol{x}_k - \frac{1}{L}\nabla f(\boldsymbol{x}_k)\Big) - f^* - \Big(f(\boldsymbol{y}_k) - f^*\Big)$$
$$= \delta_{k+1} - \delta_k \tag{12}$$
$$\nabla f(\boldsymbol{x}_k) \overset{(1)}{=} -L(\boldsymbol{y}_{k+1} - \boldsymbol{x}_k) \tag{13}$$
$$\|\nabla f(\boldsymbol{x}_k)\|_2^2 \overset{(13)}{=} L^2\|\boldsymbol{y}_{k+1} - \boldsymbol{x}_k\|_2^2 \tag{14}$$

- Put (11,13,14) into (8)

$$\delta_{k+1} \le -\frac{L}{2}\|\boldsymbol{y}_{k+1} - \boldsymbol{x}_k\|_2^2 - L\langle \boldsymbol{y}_{k+1} - \boldsymbol{x}_k, \boldsymbol{x}_k - \boldsymbol{x}^*\rangle. \tag{15}$$

- Put (12,13,14) into (9)

$$\delta_{k+1} - \delta_k \le -\frac{L}{2}\|\boldsymbol{y}_{k+1} - \boldsymbol{x}_k\|_2^2 - L\langle \boldsymbol{y}_{k+1} - \boldsymbol{x}_k, \boldsymbol{x}_k - \boldsymbol{y}_k\rangle. \tag{16}$$

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

Proof 3/6 **Stage 3: form telescoping sum**

$$
\begin{aligned}
\lambda_k &= \tfrac{1}{2}\big(1+\sqrt{1+4\lambda_{k-1}^2}\big) && (4)\\
\delta_{k+1} &\le -\tfrac{L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \boldsymbol{x}_k-\boldsymbol{x}^*\rangle && (15)\\
\delta_{k+1}-\delta_k &\le -\tfrac{L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \boldsymbol{x}_k-\boldsymbol{y}_k\rangle && (16)
\end{aligned}
$$

- **Tricky step**: consider (15) + (λ_k-1)(16).

  Left-hand size of (15) + (λ_k-1)(16) $= \delta_{k+1} + (\lambda_k-1)(\delta_{k+1}-\delta_k) = \lambda_k\delta_{k+1} - (\lambda_k-1)\delta_k$.

- Right-hand side of (15) + (λ_k-1)(16)

$$
\begin{aligned}
& -\frac{L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \boldsymbol{x}_k-\boldsymbol{x}^*\Big\rangle + (\lambda_k-1)\left(\frac{-L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \boldsymbol{x}_k-\boldsymbol{y}_k\Big\rangle\right)\\
= & -\frac{\lambda_k L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \boldsymbol{x}_k-\boldsymbol{x}^*+(\lambda_k-1)(\boldsymbol{x}_k-\boldsymbol{y}_k)\Big\rangle\\
= & -\frac{\lambda_k L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\Big\rangle
\end{aligned}
$$

- By LHS = RHS $\lambda_k\delta_{k+1} - (\lambda_k-1)\delta_k \le -\frac{\lambda_k L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\Big\rangle$.

  Multiply the inequality with λ_k:

$$
\begin{aligned}
\lambda_k^2\delta_{k+1} - \lambda_k(\lambda_k-1)\delta_k &\le -\frac{\lambda_k^2 L}{2}\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 - \lambda_k L\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\Big\rangle\\
&= -\frac{L}{2}\left(\lambda_k^2\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 + 2\lambda_k\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\Big\rangle\right). \quad (\#)
\end{aligned}
$$

- (4) gives $(2\lambda_k-1)^2 = 1+4\lambda_{k-1}^2 \iff 4\lambda_k^2-4\lambda_k+1 = 1+4\lambda_{k-1}^2 \iff \lambda_{k-1}^2 = \lambda_k(\lambda_k-1)$, put this into (#) gives

$$
\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k \le -\frac{L}{2}\left(\lambda_k^2\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 + 2\lambda_k\Big\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\Big\rangle\right) \quad (17)
$$

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

Proof 4/6

$$\lambda_k = \tfrac{1}{2}\left(1+\sqrt{1+4\lambda_{k-1}^2}\right) \tag{4}$$

$$\lambda_k^2\delta_{k+1} - \lambda_k(\lambda_k-1)\delta_k \leq -\frac{L}{2}\left(\lambda_k^2\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 + 2\lambda_k\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\rangle\right) \tag{17}$$

- Inspecting the inner product in (17) we see that it is completing squares (Thanks to Tony Silveti-Falls for figuring it out, 2023 Nov 3).

$$\|\lambda\boldsymbol{a}+\boldsymbol{b}\|_2^2 = \lambda^2\|\boldsymbol{a}\|_2^2 + 2\lambda\langle\boldsymbol{a},\boldsymbol{b}\rangle + \|\boldsymbol{b}\|_2^2 \iff \lambda^2\|\boldsymbol{a}\|_2^2 + 2\lambda\langle\boldsymbol{a},\boldsymbol{b}\rangle = \|\lambda\boldsymbol{a}+\boldsymbol{b}\|_2^2 - \|\boldsymbol{b}\|_2^2.$$

$$\begin{aligned}
&\lambda_k^2\|\boldsymbol{y}_{k+1}-\boldsymbol{x}_k\|_2^2 + 2\lambda_k\langle \boldsymbol{y}_{k+1}-\boldsymbol{x}_k,\ \lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\rangle \\
= &\left\|\lambda(\boldsymbol{y}_{k+1}-\boldsymbol{x}_k)+\lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2 - \left\|\lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2 \\
= &\left\|\lambda_k\boldsymbol{y}_{k+1}-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2 - \left\|\lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2.
\end{aligned}$$

- Using this (17) becomes

$$\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k \leq -\frac{L}{2}\left(\left\|\lambda_k\boldsymbol{y}_{k+1}-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2 - \left\|\lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2\right). \tag{18}$$

- We have $\lambda_k\boldsymbol{x}_k-(\lambda_k-1)\boldsymbol{y}_k = (1-\lambda_{k-1})\boldsymbol{y}_{k-1}+\lambda_{k-1}\boldsymbol{y}_k$.

Proof: $\gamma_k \overset{(3)}{=} \frac{1-\lambda_k}{\lambda_{k+1}} \iff \gamma_k\lambda_{k+1} = 1-\lambda_k$.

By (2) $\boldsymbol{x}_{k+1} = (1-\gamma_k)\boldsymbol{y}_{k+1}+\gamma_k\boldsymbol{y}_k$ gives $x_{k+1} = \boldsymbol{y}_{k+1}+\gamma_k(\boldsymbol{y}_k-\boldsymbol{y}_{k+1})$, multiply with λ_{k+1} gives
$\lambda_{k+1}x_{k+1} = \lambda_{k+1}\boldsymbol{y}_{k+1}+\lambda_{k+1}\gamma_k(\boldsymbol{y}_k-\boldsymbol{y}_{k+1}) = \lambda_{k+1}\boldsymbol{y}_{k+1}+(1-\lambda_k)(\boldsymbol{y}_k-\boldsymbol{y}_{k+1})$, rearrange gives
$\lambda_{k+1}x_{k+1}-\lambda_{k+1}\boldsymbol{y}_{k+1} = (1-\lambda_k)(\boldsymbol{y}_k-\boldsymbol{y}_{k+1})$, add $\boldsymbol{y}_{k+1}$ on both side gives
$\lambda_{k+1}x_{k+1}-(\lambda_{k+1}-1)\boldsymbol{y}_{k+1} = (1-\lambda_k)\boldsymbol{y}_k+\lambda_k\boldsymbol{y}_{k+1}$. Move counter k by -1 gives the result.

So (18) becomes

$$\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k \leq -\frac{L}{2}\left(\left\|\lambda_k\boldsymbol{y}_{k+1}-(\lambda_k-1)\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2 - \left\|(1-\lambda_{k-1})\boldsymbol{y}_{k-1}+\lambda_{k-1}\boldsymbol{y}_k-\boldsymbol{x}^*\right\|_2^2\right).$$

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

Proof ... 5/6

We have $\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k \;\leq\; -\frac{L}{2}\left(\|\lambda_k \boldsymbol{y}_{k+1} - (\lambda_k - 1)\boldsymbol{y}_k - \boldsymbol{x}^*\|_2^2 - \|(1-\lambda_{k-1})\boldsymbol{y}_{k-1} + \lambda_{k-1}\boldsymbol{y}_k - \boldsymbol{x}^*\|_2^2\right)$.

Rearrange the second term to make the terms in right-hand side have similar form

$$\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k \;\leq\; -\frac{L}{2}\left(\|\lambda_k \boldsymbol{y}_{k+1} - (\lambda_k - 1)\boldsymbol{y}_k - \boldsymbol{x}^*\|_2^2 - \|\lambda_{k-1}\boldsymbol{y}_k - (\lambda_{k-1} - 1)\boldsymbol{y}_{k-1} - \boldsymbol{x}^*\|_2^2\right). \tag{19}$$

Let $\boldsymbol{u}_k = \lambda_k \boldsymbol{y}_{k+1} - (\lambda_k - 1)\boldsymbol{y}_k - \boldsymbol{x}^*$ so $\lambda_{k-1}\boldsymbol{y}_k - (\lambda_{k-1} - 1)\boldsymbol{y}_{k-1} - \boldsymbol{x}^* = \boldsymbol{u}_{k-1}$ and (19) becomes

$$\begin{aligned}
\lambda_k^2\delta_{k+1} - \lambda_{k-1}^2\delta_k &\leq -\tfrac{L}{2}\left(\|\boldsymbol{u}_k\|_2^2 - \|\boldsymbol{u}_{k-1}\|_2^2\right) \\
\lambda_1^2\delta_2 - \lambda_0^2\delta_1 &\leq -\tfrac{L}{2}\left(\|\boldsymbol{u}_1\|_2^2 - \|\boldsymbol{u}_0\|_2^2\right) && \text{case } k = 1 \\
\lambda_2^2\delta_3 - \lambda_1^2\delta_2 &\leq -\tfrac{L}{2}\left(\|\boldsymbol{u}_2\|_2^2 - \|\boldsymbol{u}_1\|_2^2\right) && \text{case } k = 2 \\
&\vdots \\
\lambda_{K-1}^2\delta_K - \lambda_{K-2}^2\delta_{K-1} &\leq -\tfrac{L}{2}\left(\|\boldsymbol{u}_{K-1}\|_2^2 - \|\boldsymbol{u}_{K-2}\|_2^2\right) && \text{case } k = K-1 \\
\lambda_{K-1}^2\delta_K - \lambda_0^2\delta_1 &\leq -\tfrac{L}{2}\left(\|\boldsymbol{u}_{K-1}\|_2^2 - \|\boldsymbol{u}_0\|_2^2\right) && \text{sum } k = 1 \text{ to } k = K-1 \\
&= \tfrac{L}{2}\left(\|\boldsymbol{u}_0\|_2^2 - \|\boldsymbol{u}_{K-1}\|_2^2\right) \\
&\leq \tfrac{L}{2}\|\boldsymbol{u}_0\|_2^2 && \|\boldsymbol{u}_{K-1}\|_2^2 \geq 0
\end{aligned}$$

By definition, $\lambda_0 = 0$, $\boldsymbol{y}_0 = \boldsymbol{x}_0$, $\boldsymbol{u}_0 = \lambda_0\boldsymbol{y}_1 - (\lambda_0 - 1)\boldsymbol{y}_0 - \boldsymbol{x}^* \overset{\lambda_0=0}{=} \boldsymbol{y}_0 - \boldsymbol{x}^* \overset{\boldsymbol{y}_0=\boldsymbol{x}_0}{=} \boldsymbol{x}_0 - \boldsymbol{x}^*$, thus

$$\lambda_{K-1}^2\delta_K \leq \frac{L}{2}\|\boldsymbol{x}_0 - \boldsymbol{x}^*\|_2^2 \quad\Longrightarrow\quad \delta_K \leq \frac{L\|\boldsymbol{x}_0 - \boldsymbol{x}^*\|_2^2}{2\lambda_{K-1}^2}.$$

A typical convergence proof: Nesterov accelerated gradient

(Andersen Ang `https://angms.science/doc/CVX/CVX_NAGD.pdf`)

Proof ... 6/6

Lemma. $\lambda_{k-1} \geq \frac{k}{2}$.
Proof (by induction)

- Case $k = 0$ and $\lambda_0 = 0$. It is trivial $0 \geq 0/2$.
- Case $k = 1$. By definition,
  $\lambda_k = \frac{1+\sqrt{1+4\lambda_{k-1}^2}}{2} = \frac{1+\sqrt{1+4\cdot 0^2}}{2} = 1 > \frac{1}{2} = \frac{k}{2}\Big|_{k=1}$
- Induction hypothesis: assume $\lambda_{n-1} \geq \frac{n}{2}$.
- Case $k = n$

$$\begin{aligned}
\lambda_n &= \frac{1+\sqrt{1+4\lambda_{n-1}^2}}{2} \\
&\geq \frac{1+\sqrt{1+4\left(\frac{n}{2}\right)^2}}{2} \quad \text{[Induction hypothesis]} \\
&= \frac{1+\sqrt{1+n^2}}{2} \\
&> \frac{1+\sqrt{n^2}}{2} \\
&= \frac{1+n}{2}. \quad \square
\end{aligned}$$

With $\lambda_{k-1} \geq \frac{k}{2}$, so

$$\frac{1}{\lambda_{k-1}^2} \leq \frac{4}{k^2}.$$

Therefore $\delta_K \leq \frac{L\|\boldsymbol{x}_0 - \boldsymbol{x}^*\|_2^2}{2\lambda_{K-1}^2}$ becomes

$$f(\boldsymbol{y}_K) - f^* \leq \frac{2L\|\boldsymbol{x}_0 - \boldsymbol{x}^*\|_2^2}{K^2}.$$

where $f(\boldsymbol{y}_K) - f^* =: \delta_K$. $\quad \square$

Rename K as k gives

$$f(\boldsymbol{y}_k) - f^* \leq \frac{2L\|x_0 - \boldsymbol{x}^*\|_2^2}{k^2}.$$

This $\begin{cases} \text{complicated} \\ \text{highly-involved} \\ \text{non-intuitive} \end{cases}$ proof is now completed.

Outline

Performance estimation of an optimization method

Goal of a PEP (Performance Estimation Problem):
compute the exact worst-case behavior

Example:

Performance estimation of an optimization method

Goal of a PEP (Performance Estimation Problem):
compute the exact worst-case behavior

- of a given optimization black-box method
  (considering a fixed number of iterations)

Example: after computing N steps of the gradient method with fixed unit step-size $x_{k+1} = x_k - \nabla f(x_k)$

Performance estimation of an optimization method

Goal of a PEP (Performance Estimation Problem):
compute the exact worst-case behavior

- of a given optimization black-box method
  (considering a fixed number of iterations)
- applied to any function belonging to a given class
  (e.g. convex/nonconvex functions, with smoothness/strong convexity)

Example: after computing N steps of the gradient method with fixed unit step-size $x_{k+1} = x_k - \nabla f(x_k)$ applied to a convex function f with a 1-Lipschitz gradient and minimizer x^*

Performance estimation of an optimization method

Goal of a PEP (Performance Estimation Problem):
compute the exact worst-case behavior

- of a given optimization black-box method
  (considering a fixed number of iterations)
- applied to any function belonging to a given class
  (e.g. convex/nonconvex functions, with smoothness/strong convexity)
- from any starting point
  (satisfying some condition w.r.t. a minimizer)

Example: after computing N steps of the gradient method with fixed unit step-size $x_{k+1} = x_k - \nabla f(x_k)$ applied to a convex function f with a 1-Lipschitz gradient and minimizer x^* from an initial iterate satisfying $\|x_0 - x_*\| \leq 1$,

Performance estimation of an optimization method

Goal of a PEP (Performance Estimation Problem):
compute the exact worst-case behavior

- of a given optimization black-box method
  (considering a fixed number of iterations)
- applied to any function belonging to a given class
  (e.g. convex/nonconvex functions, with smoothness/strong convexity)
- from any starting point
  (satisfying some condition w.r.t. a minimizer)
- for a given performance criteria
  (e.g. objective accuracy, distance to solution, gradient norm)

Example: after computing N steps of the gradient method with fixed unit step-size $x_{k+1} = x_k - \nabla f(x_k)$ applied to a convex function f with a 1-Lipschitz gradient and minimizer x^* from an initial iterate satisfying $\|x_0 - x_*\| \leq 1$, what is the worst (largest) possible objective function accuracy for the last iterate $f(x_N) - f(x_*)$?

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

- compute the exact value of the performance criteria's worst-case = optimal value of PEP problem

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

- compute the exact value of the performance criteria's worst-case = optimal value of PEP problem
- identify an explicit function (and starting point) achieving this worst-case value = primal solution of PEP problem + interpolation

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

- compute the exact value of the performance criteria's worst-case = optimal value of PEP problem
- identify an explicit function (and starting point) achieving this worst-case value = primal solution of PEP problem + interpolation
- obtain an independently-checkable proof that this worst-case value is a valid (upper) bound on the performance criteria = dual multiplier of PEP problem

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

- compute the exact value of the performance criteria's worst-case = optimal value of PEP problem
- identify an explicit function (and starting point) achieving this worst-case value = primal solution of PEP problem + interpolation
- obtain an independently-checkable proof that this worst-case value is a valid (upper) bound on the performance criteria = dual multiplier of PEP problem
- all three steps can be done either numerically or analytically

Output of a performance estimation problem

For a given PEP (Performance Estimation Problem) we will

- compute the exact value of the performance criteria's worst-case = optimal value of PEP problem
- identify an explicit function (and starting point) achieving this worst-case value = primal solution of PEP problem + interpolation
- obtain an independently-checkable proof that this worst-case value is a valid (upper) bound on the performance criteria = dual multiplier of PEP problem
- all three steps can be done either numerically or analytically

For a large class of first-order methods applied to convex compositive optimization problems, these can be computed exactly using a semidefinite programming (SDP) problem

Very brief history of performance estimation

- Drori and Teboulle introduce performance estimation (2012, published 2014) and solve a relaxation of a nonconvex formulation, providing bounds shown to be tight for some algorithms (with matching worst-case functions)

Very brief history of performance estimation

- Drori and Teboulle introduce performance estimation (2012, published 2014) and solve a relaxation of a nonconvex formulation, providing bounds shown to be tight for some algorithms (with matching worst-case functions)

- Taylor, Hendrickx and Glineur introduce an exact SDP formulation (2015) relyling on the concept of necessary and sufficient interpolation conditions

Very brief history of performance estimation

- Drori and Teboulle introduce performance estimation (2012, published 2014) and solve a relaxation of a nonconvex formulation, providing bounds shown to be tight for some algorithms (with matching worst-case functions)

- Taylor, Hendrickx and Glineur introduce an exact SDP formulation (2015) relyling on the concept of necessary and sufficient interpolation conditions

- Other approaches similar in spirit:
  integral quadratic constraints by Lessard, Recht, Packard (2014);
  Lyapunov/potential functions by Taylor, Bach (2019)

Very brief history of performance estimation

- Drori and Teboulle introduce performance estimation (2012, published 2014) and solve a relaxation of a nonconvex formulation, providing bounds shown to be tight for some algorithms (with matching worst-case functions)

- Taylor, Hendrickx and Glineur introduce an exact SDP formulation (2015) relyling on the concept of necessary and sufficient interpolation conditions

- Other approaches similar in spirit:
  integral quadratic constraints by Lessard, Recht, Packard (2014);
  Lyapunov/potential functions by Taylor, Bach (2019)

- A growing subfield of algorithmic optimization (above three seminal papers total 1700+ citations)

Outline

Obtaining a SDP formulation: a worked example

We explain all steps to obtain an exact SDP formulation on the following example, dealing with the gradient method with the simplest $\frac{1}{L}$ steps:

What is the worst possible objective function accuracy after applying N steps of the gradient method with fixed step-size $\frac{1}{L}$ to a convex function f with L-Lipschitz gradient and minimizer x_*, from an initial iterate satisfying $\|x_0 - x_*\| \leq R$?

Obtaining a SDP formulation: a worked example

We explain all steps to obtain an exact SDP formulation on the following example, dealing with the gradient method with the simplest $\frac{1}{L}$ steps:

> What is the worst possible objective function accuracy after applying N steps of the gradient method with fixed step-size $\frac{1}{L}$ to a convex function f with L-Lipschitz gradient and minimizer x_*, from an initial iterate satisfying $\|x_0 - x_*\| \leq R$?

Formally, this is an infinite-dimensional problem over functions f:

$$\max_{f, x_*, x_0, x_1, \ldots, x_N} f(x_N) - f(x_*) \text{ s.t. } \begin{cases} f \text{ is proper and convex} \\ \text{f has an } L\text{-Lipschitz gradient} \\ x_* \text{ is a minimizer of } f \\ \|x_0 - x_*\| \leq R \\ x_{i+1} = x_i - \frac{1}{L} g_i \text{ for every } 0 \leq i < N \\ (\text{where } g_i = \nabla f(x_i) \text{ is the gradient}) \end{cases}$$

Step 1: use the black-box property

Performance estimation problem is infinite dimensional,
but the gradient method is a black-box optimization method

Next iterate only depends on previously obtained oracle information, consisting of the function value and an (arbitrary) subgradient for each of the previous iterates

In general, starting from initial x_0, a black-box method computes

$$\begin{aligned} x_1 &= \mathcal{M}_1\left(x_0, \mathcal{O}_f(x_0)\right), \\ x_2 &= \mathcal{M}_2\left(x_0, \mathcal{O}_f(x_0), \mathcal{O}_f(x_1)\right), \\ &\vdots \\ x_N &= \mathcal{M}_N\left(x_0, \mathcal{O}_f(x_0), \ldots, \mathcal{O}_f(x_{N-1})\right). \end{aligned}$$

All iterates depend only on x_0 and the finite list of outputs from the oracle $(\mathcal{O}_f(x_i) = \{f(x_i), \nabla f(x_i)\})$

Step 1: finite-dimensional reformulation using black-box

Hence infinite-dimensional variable f can be replaced by the list of oracle outputs, each defining a variable

$$f_i = f(x_i) \text{ and } g_i = \nabla f(x_i) \text{ for all } i = 0, 1, \dots N \text{ as well as } i = *$$

and performance estimation problem becomes

$$\begin{cases} \max \\ x_*, x_0, x_1, \dots, x_N \\ f_*, f_0, f_1, \dots, f_N \\ g_*, g_0, g_1, \dots, g_N \end{cases} f_N - f_* \text{ s.t.} \begin{cases} \text{there exists proper and convex } f \text{ with} \\ \text{an } L\text{-Lipschitz gradient such that} \\ \qquad f(x_i) = f_i \text{ and } g_i = \nabla f(x_i) \\ \text{for all } i \in \{*, 0, 1, \dots N\} \\ g_* = 0 \\ \|x_0 - x_*\| \leq R \\ x_{i+1} = x_i - \frac{1}{L} g_i \text{ for every } 0 \leq i < N \end{cases}$$

which is finite-dimensional (minimizer condition became $g_* = 0$).
Remaining issue: imposing existence of suitable f compatible with f_i and g_i requires necessary and sufficient interpolation conditions

Step 2: introduce interpolation conditions

We need to express in our problem the fact that

> there exists proper and convex f with an L-Lipschitz gradient such that $f(x_i) = f_i$ and $g_i = \nabla f(x_i)$ for all $i \in I = \{*, 0, 1, \dots N\}$

i.e. given values of x_i, f_i and g_i we must guarantee existence of f

$\rightarrow$ we need necessary and sufficient conditions for interpolation

Step 2: introduce interpolation conditions

We need to express in our problem the fact that

> there exists proper and convex f with an L-Lipschitz gradient such that $f(x_i) = f_i$ and $g_i = \nabla f(x_i)$ for all $i \in I = \{*, 0, 1, \dots N\}$

i.e. given values of x_i, f_i and g_i we must guarantee existence of f

$\rightarrow$ we need necessary and sufficient conditions for interpolation

We use the following equivalence

there exists proper and convex f satisfying
$f(x_i) = f_i$ and $g_i = \nabla f(x_i)$ for every $i \in I$

$$\Leftrightarrow$$

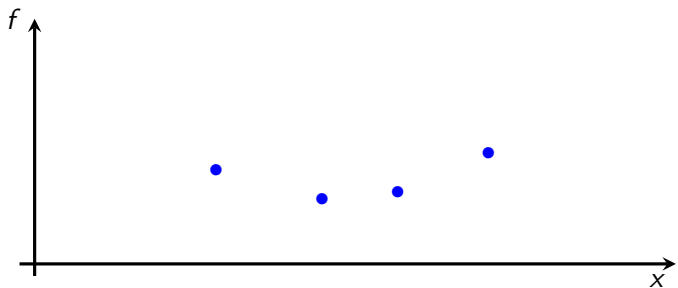
$f_j \geq f_i + g_i(x_j - x_i) + \frac{1}{2L}\|g_i - g_j\|^2$ for every pair $i \in I, j \in I$

(can be extended to deal with the smooth strongly convex case, and the smooth nonconvex/weakly convex cases)

Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

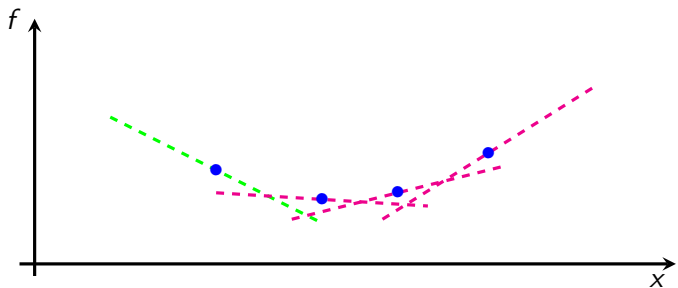
When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?



Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

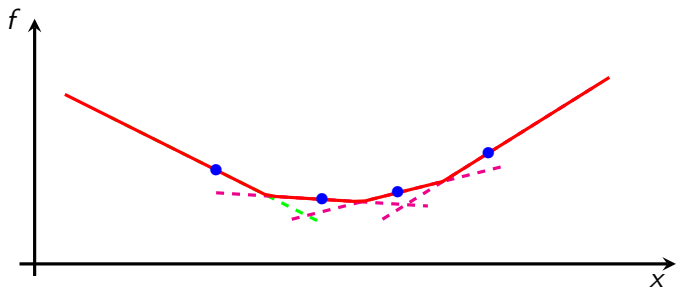
When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?



Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

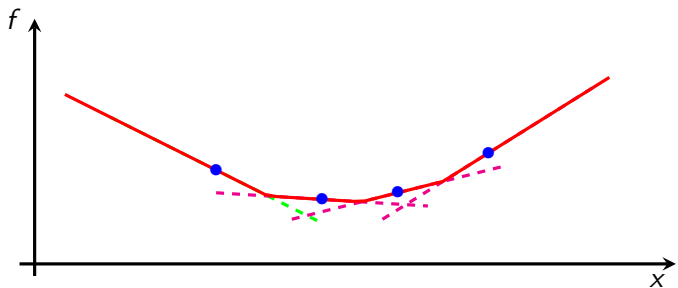
When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?



Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?

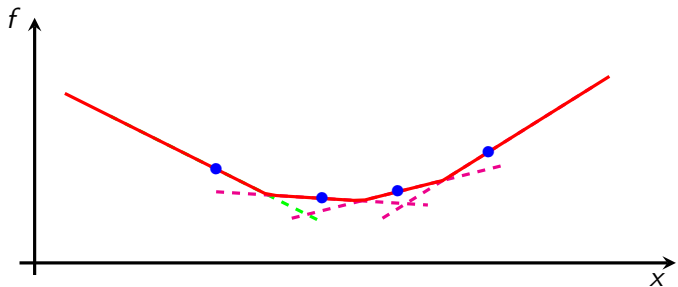


Subgradient inequalities $f_i \geq f_j + g_j^T (x_i - x_j)\ \forall i, j \in S$ are necessary

Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?



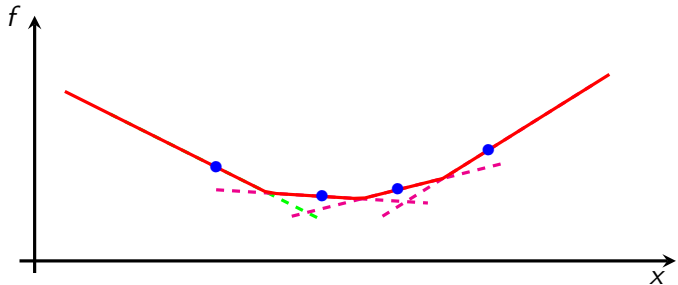
Subgradient inequalities $f_i \geq f_j + g_j^T(x_i - x_j)\ \forall i, j \in S$ are necessary

Function $f(x) = \max_j \left\{ f_j + g_j^T(x - x_j) \right\}$ is then an interpolating function (piecewise linear, not unique!)

Necessary and sufficient conditions for convex interpolation

First with a **simpler** class: convex functions

When is set $\{(x_i, g_i, f_i)\}_{i \in S}$ interpolable by a convex function?



Subgradient inequalities $f_i \geq f_j + g_j^T(x_i - x_j)\ \forall i, j \in S$ are necessary

Function $f(x) = \max_j \left\{ f_j + g_j^T(x - x_j) \right\}$ is then an interpolating function (piecewise linear, not unique!)

Hence subgradient inequalities are necessary and sufficient for interpolation

Smooth strongly convex interpolation

We want to handle interpolation by L-smooth μ-strongly convex functions ($\mathcal{F}_{\mu,L}$)

Key idea: reduction to convex interpolation

Smooth strongly convex interpolation

We want to handle interpolation by L-smooth μ-strongly convex functions ($\mathcal{F}_{\mu,L}$)

Key idea: reduction to convex interpolation

Using two basic operations to transform the problem:

Smooth strongly convex interpolation

We want to handle interpolation by L-smooth μ-strongly convex functions ($\mathcal{F}_{\mu,L}$)

Key idea: reduction to convex interpolation

Using two basic operations to transform the problem:

- Conjugation: f is closed, proper and convex, then:
  f L-Lipschitz gradient $\Leftrightarrow$ f^* $\frac{1}{L}$-strongly convex

$$(x_i, g_i, f_i)_{i\in S} \text{ is } \mathcal{F}_{0,L}\text{-interp.} \Leftrightarrow (g_i, x_i, x_i^T g_i - f_i)_{i\in S} \text{ is } \mathcal{F}_{1/L,\infty}\text{-interp.}$$

Smooth strongly convex interpolation

We want to handle interpolation by L-smooth μ-strongly convex functions ($\mathcal{F}_{\mu,L}$)

Key idea: reduction to convex interpolation

Using two basic operations to transform the problem:

- Conjugation: f is closed, proper and convex, then:
  f L-Lipschitz gradient $\Leftrightarrow$ f^* $\frac{1}{L}$-strongly convex

$$(x_i, g_i, f_i)_{i\in S} \text{ is } \mathcal{F}_{0,L}\text{-interp.} \Leftrightarrow (g_i, x_i, x_i^T g_i - f_i)_{i\in S} \text{ is } \mathcal{F}_{1/L,\infty}\text{-interp.}$$

- Minimal curvature subtraction:
  $f(x)$ μ-strongly convex $\Leftrightarrow$ $f(x) - \frac{\mu}{2}||x||_2^2$ convex

Since $\nabla\left(f(x) - \frac{\mu}{2}||x||^2\right) = \nabla f(x) - \mu x$ we have

$$(x_i, g_i, f_i)_{i\in S} \text{ is } \mathcal{F}_{\mu,L}\text{-interpolable}$$

$$\Leftrightarrow (x_i, g_i - \mu x_i, f_i - \frac{\mu}{2}||x_i||^2)_{i\in S} \text{ is } \mathcal{F}_{0,L-\mu}\text{-interpolable}$$

Smooth strongly convex interpolation conditions

Theorem

Set $\{(x_i, g_i, f_i)\}_{i \in S}$ is $\mathcal{F}_{\mu,L}$-interpolable if and only

$$f_i - f_j - g_j^\top (x_i - x_j) \geq \frac{1}{2(1-\mu/L)} \left(\frac{1}{L} \|g_i - g_j\|_2^2 + \mu \|x_i - x_j\|_2^2 - 2\frac{\mu}{L} (g_j - g_i)^\top (x_j - x_i) \right)$$

holds for every pair of indices $i \in I$ and $j \in S$

Condition also works for negative μ (nonconvex functions)

Smooth strongly convex interpolation conditions

Theorem

Set $\{(x_i, g_i, f_i)\}_{i \in S}$ is $\mathcal{F}_{\mu,L}$-interpolable if and only

$$f_i - f_j - g_j^\top (x_i - x_j) \geq \frac{1}{2(1-\mu/L)} \left(\frac{1}{L} \|g_i - g_j\|_2^2 + \mu \|x_i - x_j\|_2^2 - 2\frac{\mu}{L} (g_j - g_i)^\top (x_j - x_i) \right)$$

holds for every pair of indices $i \in I$ and $j \in S$

When $\mu = 0$ (L-smooth functions), those conditions reduce to

$$f_j \geq f_i + g_i^T (x_j - x_i) + \frac{1}{2L} ||g_i - g_j||_2^2 \qquad \forall i, j \in S$$

Condition also works for negative μ (nonconvex functions)

Smooth strongly convex interpolation conditions

Theorem

Set $\{(x_i, g_i, f_i)\}_{i \in S}$ is $\mathcal{F}_{\mu,L}$-interpolable if and only

$$f_i - f_j - g_j^\top (x_i - x_j) \geq \frac{1}{2(1-\mu/L)} \left(\frac{1}{L} \|g_i - g_j\|_2^2 + \mu \|x_i - x_j\|_2^2 - 2\frac{\mu}{L} (g_j - g_i)^\top (x_j - x_i) \right)$$

holds for every pair of indices $i \in I$ and $j \in S$

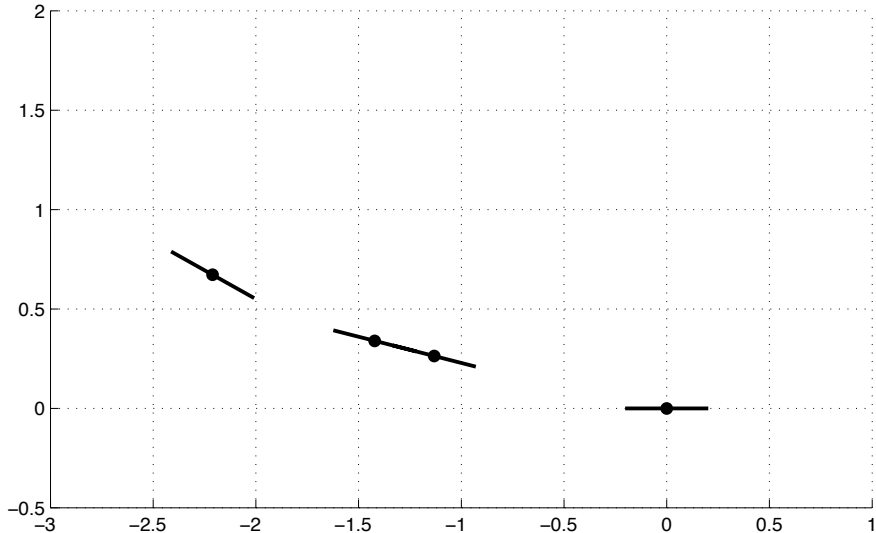
When $\mu = 0$ (L-smooth functions), those conditions reduce to

$$f_j \geq f_i + g_i^T (x_j - x_i) + \frac{1}{2L} ||g_i - g_j||_2^2 \qquad \forall i, j \in S$$

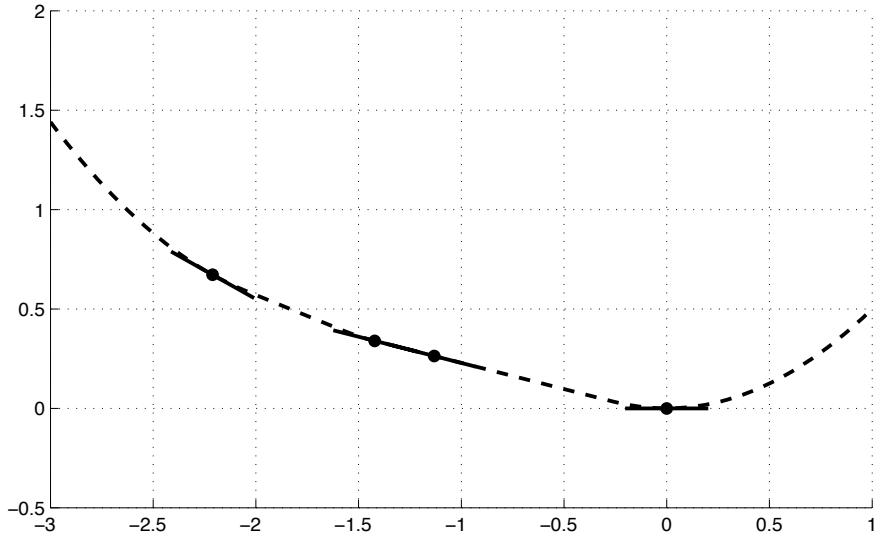
Condition also works for negative μ (nonconvex functions)

Proof is constructive, provides an explicit interpolating function (piecewise linear-quadratic, not unique)

Example



Example



Step 2 (cont.): introduce interpolation conditions

Performance estimation problem becomes
(with $I = \{*, 0, 1, \dots, N\}$)

$$\max_{\{x_i, f_i, g_i\}_{i \in I}} f_N - f_* \text{ s.t. } \begin{cases} f_j \geq f_i + g_i^T (x_j - x_i) + \frac{1}{2L} \|g_i - g_j\|^2 \ \forall i \neq j \in I \\ x_* = 0, f_* = 0, g_* = 0 \\ \|x_0 - x_*\| \leq R \\ x_{i+1} = x_i - \frac{1}{L} g_i \text{ for every } 0 \leq i < N \end{cases}$$

(note that constraints $f_* = 0$ and $x* = 0$ can be added w.l.o.g.)

Worst-case is computed exactly with a finite explicit formulation
idea first introduced in Drori and Teboulle (2014)

Unfortunately, problem is nonconvex (inner products $g_i^T x_j$)

Step 3: convexify using a Gram matrix

$$\max_{\{x_i,f_i,g_i\}_{i\in I}} f_N - f_* \text{ s.t. } \begin{cases} f_j \geq f_i + g_i^T(x_j - x_i) + \frac{1}{2L}\|g_i - g_j\|^2 \ \forall i \neq j \in I \\ x_* = 0, f_* = 0, g_* = 0 \\ \|x_0 - x_*\|^2 \leq R^2 \\ \left\|x_{i+1} - x_i + \frac{1}{L}g_i\right\|^2 = 0 \text{ for every } 0 \leq i < N \end{cases}$$

All terms are quadratic in x_i and g_i, and linear in f_i

$\rightarrow$ introduce a Gram matrix for variables f_i and g_i !

Step 3: convexify using a Gram matrix

$$\max_{\{x_i,f_i,g_i\}_{i\in I}} f_N - f_* \text{ s.t. } \begin{cases} f_j \geq f_i + g_i^T(x_j - x_i) + \frac{1}{2L}\|g_i - g_j\|^2 \ \forall i \neq j \in I \\ x_* = 0, f_* = 0, g_* = 0 \\ \|x_0 - x_*\|^2 \leq R^2 \\ \left\|x_{i+1} - x_i + \frac{1}{L}g_i\right\|^2 = 0 \text{ for every } 0 \leq i < N \end{cases}$$

All terms are quadratic in x_i and g_i, and linear in f_i
$\rightarrow$ introduce a Gram matrix for variables f_i and g_i !

$$\text{Example } N = 1 : \qquad P = (x_0 \mid x_1 \mid g_0 \mid g_1)$$

$$G = P^T P = \begin{pmatrix} x_0^T x_0 & x_0^T x_1 & x_0^T g_0 & x_0^T g_1 \\ x_1^T x_0 & x_1^T x_1 & x_1^T g_0 & x_1^T g_1 \\ g_0^T x_0 & g_0^T x_1 & g_0^T g_0 & g_0^T g_1 \\ g_1^T x_0 & g_1^T x_1 & g_1^T g_0 & g_1^T g_1 \end{pmatrix} \succeq 0.$$

Step 3: convexify using a Gram matrix

Problem becomes linear in f_i and elements of the positive semidefinite Gram matrix

$\rightarrow$ a semidefinite optimization problem

Convex problem, can be solved globally and efficiently with interior-point methods

Step 3: convexify using a Gram matrix

Problem becomes linear in f_i and elements of the positive semidefinite Gram matrix
$\rightarrow$ a semidefinite optimization problem
Convex problem, can be solved globally and efficiently with interior-point methods

Formulation is a priori exact if the Gram matrix reformulation is equivalent,
which happens if dimension of vectors x_i and g_i (i.e. dimension of f) is large enough
(larger than dimension of G)

$\rightarrow$ an exact dimension-free formulation for the large-scale case
(for results valid for small-dimensional f, impose rank constraint on G ; worst-case can only deteriorate as dimension of f increases)

Step 3: convexify using a Gram matrix

Problem becomes linear in f_i and elements of the positive semidefinite Gram matrix
$\rightarrow$ a semidefinite optimization problem
Convex problem, can be solved globally and efficiently with interior-point methods

Formulation is a priori exact if the Gram matrix reformulation is equivalent,
which happens if dimension of vectors x_i and g_i (i.e. dimension of f) is large enough
(larger than dimension of G)

$\rightarrow$ an exact dimension-free formulation for the large-scale case
(for results valid for small-dimensional f, impose rank constraint on G ; worst-case can only deteriorate as dimension of f increases)

From $G \succeq 0$, we can recover values of x_i and g_i corresponding to compute worst-case function ; combined with values f_i this allows to identify an explicit function achieving the worst-case, using interpolation

Rank of G determines dimension of worst-case f (at most $2N + 2$)

Step 4: recovering outputs

- Exact worst-case value computed by the SDP formulation
  (a priori under the large-scale assumption)

Step 4: recovering outputs

- Exact worst-case value computed by the SDP formulation
  (a priori under the large-scale assumption)
- Independently-checkable proof is derived from dual solution:
  combining interpolation inequalities with the corresponding dual multipliers
  provides a proof for the worst-case bound (see later)

Step 4: recovering outputs

- Exact worst-case value computed by the SDP formulation
  (a priori under the large-scale assumption)
- Independently-checkable proof is derived from dual solution:
  combining interpolation inequalities with the corresponding dual multipliers
  provides a proof for the worst-case bound (see later)
- Explicit worst-case function derived by interpolation from optimal solution,
  provides a function whose dimension is equal to rank of G
- Worst-case result valid for all dimensions larger than that rank of optimal G
  In particular: if G has rank one, worst-case unconditionally valid,
  which is surprisingly frequent

Additional step: use homogeneity

Our performance estimation problem is parameterized by

- Lipschitz constant L for gradient
- Maximum distance R between starting point and a minimizer

Additional step: use homogeneity

Our performance estimation problem is parameterized by

- Lipschitz constant L for gradient
- Maximum distance R between starting point and a minimizer

To avoid having to solve for every value of L and R use homogeneity properties

- if $L \to \lambda L$ with $\lambda > 0$, worst-case is also multiplied by λ (proof: scale $f \to \lambda f$)
- if $R \to \lambda R$ with $\lambda > 0$, worst-case is also multiplied by λ^2 (proof: $f \to \lambda^2 \lambda f(\cdot/\lambda)$)

hence worst-case $w_*(L, R, N)$ is proportional to L and R^2

Additional step: use homogeneity

Our performance estimation problem is parameterized by

- Lipschitz constant L for gradient
- Maximum distance R between starting point and a minimizer

To avoid having to solve for every value of L and R use homogeneity properties

- if $L \to \lambda L$ with $\lambda > 0$, worst-case is also multiplied by λ (proof: scale $f \to \lambda f$)
- if $R \to \lambda R$ with $\lambda > 0$, worst-case is also multiplied by λ^2 (proof: $f \to \lambda^2 \lambda f(\cdot/\lambda)$)

hence worst-case $w_*(L, R, N)$ is proportional to L and R^2

$\to$ solve problem for $B = R = 1$, find worst-case value $w_*(1, 1, N)$ so that for general L and R we will have

$$f(x_N) - f(x_*) \leq w_*(L, R, N) = \frac{LR^2}{2} \cdot w_*(1, 1, N)$$

Only remaining parameter is N, number of steps

Time for some hands-on session!

Let's now use the PEPit toolbox

or visit

https://github.com/PerformanceEstimation/Tutorial-SIAM-OP26

Outline